Optimal Planning and Control: Part I

Luca Dall'Asta

DISAT
Politecnico di Torino, Italy

October 2025

Optimal planning and sequential decision problems

Optimal planning is the application of the *principle of maximum expected utility* when the expected utility function includes also the utility generated in the subsequent decision stages assuming a separable form. The current decision is taken assuming optimal decision at the subsequent stages (optimal substructure).

Optimal planning is equivalent to the problem of optimally controlling a (possibly stochastic) discrete-time dynamical system,

$$x_{t+1} = f_t\left(x_t, a_t, w_{t+1}\right), \qquad t = 0, 1, \ldots, T$$

where $x_t \in X$ is the *state* of the world at time t, $a_t \in A_t$ is the *action* (or control variable), $w_{t+1} \sim P\left(w_{t+1}|x_t, a_t\right)$ is a random parameter (or *disturbancy*).

Optimal planning and sequential decision problems

A *reward* $r_t(x_t, a_t)$ is associated to each *stage* t of the decision problem.

The total reward function is assumed to be additive,

$$R_T(x, a) = r_T(x_T) + \sum_{t=0}^{T-1} r_t(x_t, a_t)$$

with terminal reward $r_T(x_T)$.

Since states are random variables (actions are not), then the payoff functions are random variables.

The goal is to optimize the ***expected payoff/utility function***

$$\mathbb{E}_w [R_T(x, a)]$$

Optimal planning and sequential decision problems

The optimization is not over a sequence of actions $a = (a_0, a_1, \ldots, a_{T-1})$, but over a function μ that we call a ***policy*** (or contingency plan),

$$\mu_t : X \to A, \qquad \forall t = 0, 1, \ldots, T-1,$$

that assigns an action a_t to each admissible state x_t at time t.

For each policy $\mu = (\mu_0, \ldots, \mu_{T-1})$ and initial state x_0, consider the ***value function*** $V^\mu(x_0)$ defined by

$$V^\mu(x_0) \equiv \mathbb{E}_w \left[R_T \left(x, \mu(x) \right) \right] = \mathbb{E}_w \left[r_T(x_T) + \sum_{t=0}^{T-1} r_t \left(x_t, \mu_t(x_t) \right) \right],$$

the *optimal policy* is a policy μ^* such that

$$V^{\mu^*}(x_0) = \max_{\mu \in \Pi} V^\mu(x_0) \equiv V^*(x_0).$$

Optimal planning: Principle of Optimality

Proposition (Bellman's Principle of Optimality)

Let $\mu^* = (\mu_0^*, \mu_1^*, \ldots, \mu_{T-1}^*)$ be an optimal policy for the stochastic planning problem and assume that, when using μ^*, a state x_s occurs at time s with positive probability. Consider the subproblem starting at time s in state x_s for which we aim at optimizing the value-to-go

$$V^{\mu_{(s)}}(x_s) = \mathbb{E}_w \left[r_T(x_T) + \sum_{t=s}^{T-1} r_t \left(x_t, \mu_t(x_t) \right) \right]$$

where $\mu_{(s)} = (\mu_s, \mu_{s+1}, \ldots, \mu_{T-1})$. Then, the truncated policy $\mu_{(s)}^* = (\mu_s^*, \mu_{s+1}^*, \ldots, \mu_{T-1}^*)$ is optimal for the subproblem, i.e.

$$V^{\mu_{(s)}^*}(x_s) = \max_{\mu_{(s)} \in \Pi} V^{\mu_{(s)}}(x_s) \equiv V_s^*(x_s).$$

Optimal substructure is a *necessary condition* for optimal policies.

Optimal planning: Dynamic Programming algorithm

Proposition

$\forall x_0$, the optimal value $V^*(x_0)$ is equal to $V_0(x_0)$ given by the following ***Dynamic Programming*** *algorithm which proceeds backwards in time from $T-1$ to 0 according to the (discrete) Bellman equation*

$$\begin{aligned} V_T(x_T) &= r_T(x_T) \\ V_t(x_t) &= \max_{a_t \in A_t(x_t)} \mathbb{E}\left[r_t(x_t, a_t) + V_{t+1}\left(f_t(x_t, a_t, w_{t+1})\right)\right], \end{aligned}$$

for $0 \leq t < T$. If $a_t^* = \mu_t^*(x_t)$ maximizes the r.h.s. for each x_t and t, then the policy $\mu^* = (\mu_0^*, \mu_1^*, \ldots, \mu_{T-1}^*)$ is optimal.

Remark

DP algorithm requires $O(m\,|X|^2\,T)$ operations (where $m = \max_\alpha m_\alpha$, with $m_\alpha = |A(\alpha)|$). A brute force algorithm requires $O(m^{|X|\,T})$ operations.

Optimal planning: Dynamic Programming algorithm

Exercise (Optimal Consumption)

Sequential planning problem with finite horizon T in which an individual has to decide how much to consume at each period. The action at time t is the consumption level $a_t \geq 0$ and the stage utility is $u(a_t) = \log a_t$. The expected utility function is

$$U(a) = \sum_{t=0}^{T} \delta^t u(a_t), \qquad \delta < 1$$

The relation between capitals at two successive periods is

$$x_{t+1} = f(x_t) - a_t$$

with $f(x) = Ax^\alpha$ $(0 < \alpha < 1)$ being the Cobb-Douglas production function with full depreciation. We also impose $x_t \geq 0, \forall t \geq 0$.

Optimal planning in Markov chains

The formulation can be adapted to decision processes in Markov chains.

Consider a discrete state space X and the transition probability

$$p_{xx'}^{t}(a) = \Pr\left[x_{t+1} = x' | x_t = x, a_t = a\right]$$

which means $\tilde{w}_{t+1}(\{x, x'; a\}) = \{w_{t+1} | x' = f_t(x, a, w_{t+1})\}$ such that

$$\Pr\left[\tilde{w}_{t+1}(\{x, x', a\}) | x_t = x, a_t = a\right] = p_{xx'}^{t}(a).$$

The finite-horizon Bellman equation is

$$V_t(x) = \max_{a \in A} \left[r_t(x, a) + \sum_{x' \in X} p_{xx'}^{t}(a) V_{t+1}(x') \right],$$

with $V_T(x) = r_T(x)$ and the optimal policy $\mu^* = (\mu_0^*, \mu_1^*, \ldots, \mu_{T-1}^*)$ is s.t.

$$\mu_t^*(x) \in \arg\max_{a \in A} \left[r_t(x, a) + \sum_{x' \in X} p_{xx'}^{t}(a) V_{t+1}^*(x') \right], \qquad \forall t.$$

Markov Decision Process

Consider a $T = \infty$ problem, with time-independent rewards.

Definition

A ***Markov Decision Process*** (MDP) is a tuple (X, A, r, p) where

- discrete set X is the state space,
- discrete set A is the action space (can be state dependent),
- function $r : X \times A \to \mathbb{R}$ is a reward function $r(x, a)$,
- distribution $p(x'|x, a) = p_{xx'}(a)$, $\forall x'$.

Definition

In the MDP specified by the tuple (X, A, r, p), a *(stationary) policy* is a function $\mu : X \to A$ assigning an action $\mu(x) = a \in A$ at each state $x \in X$. The process $\{x_t\}_{t \geq 0}$ obtained is called a *(controlled) Markov chain*, i.e.

$$\Pr\left[x_{t+1} = x' | x_t = x\right] = \Pr\left[x_{t+1} = x' | x_t = x, a_t = \mu(x)\right] = p_{xx'}(\mu(x)).$$

Markov Decision Process: policy evaluation

The time-discounted total expected reward under a policy μ is defined as

$$V^{\mu}(x_0) = \mathbb{E}_p \left[\sum_{t=0}^{\infty} \delta^t r(x_t, \mu(x_t)) \right], \qquad 0 \leq \delta < 1$$

Given a policy μ, it satisfies the equation (**policy evaluation**)

$$V^{\mu}(x) = r(x, \mu(x)) + \delta \sum_{x' \in X} p_{xx'}(\mu(x)) V^{\mu}(x'),$$

which is linear and can be solved using linear algebra,

$$V^{\mu} = r^{\mu} + \delta P^{\mu} V^{\mu} \quad \Longrightarrow \quad V^{\mu} = (I - \delta P^{\mu})^{-1} r^{\mu},$$

with the condition that $0 < \delta < 1$ to ensure invertibility.

As an alternative it can be solved by iteration.

Markov Decision Process: optimal policy

The optimal policy solves the ***stationary Bellman equation***

$$\begin{aligned} V(x) &= \max_{a \in A(x)} \left[r(x,a) + \delta \sum_{x' \in X} p_{xx'}(a) V(x') \right] \\ &= \mathcal{B}[V](x). \end{aligned}$$

It can be solved by *iteration* (thanks to contraction properties of the Bellman operator $\mathcal{B}$ when $0 < \delta < 1$).

Common solution methods*:*

1. *Value Iteration*
2. *Policy Iteration*
3. *Linear Programming*

Markov Decision Process: optimal policy

Value Iteration algorithm:

- *Input*: convergence threshold $\epsilon > 0$, initial value $V_{(0)}(x)$, $\forall x \in X$,
- *Repeat:* while $\left\| \mathcal{B}[V_{(i)}](x) - V_{(i)}(x) \right\| > \epsilon$, $\forall x \in X$,
  - do $V_{(i+1)}(x) = \mathcal{B}[V_{(i)}](x)$, $\forall x \in X$
  - set $i \leftarrow i + 1$,
- *Output*: compute the optimal $V(x) = V_{(i)}(x)$ and $\mu(x) = \arg \mathcal{B}[V](x)$.

Remark

The error in the optimal value is at most $\delta\epsilon/(1-\delta)$.
The computational time is at most $O(|X|^2|A|)$ per iteration.

Markov Decision Process: optimal policy

Policy Iteration algorithm:

- *Input*: initial policy function $\mu_{(0)}(x)$, $\forall x \in X$, dummy policy μ',
- *Repeat:* while $\mu \neq \mu'$
  - set $\mu' \leftarrow \mu$,
  - compute $V^{\mu}(x)$, $\forall x \in X$ (evaluation step)
  - set $\mu(x) = \arg \mathcal{B}[V^{\mu}](x)$ (improvement step)
- *Output*: return policy μ and the optimal value $V = V^{\mu}$.

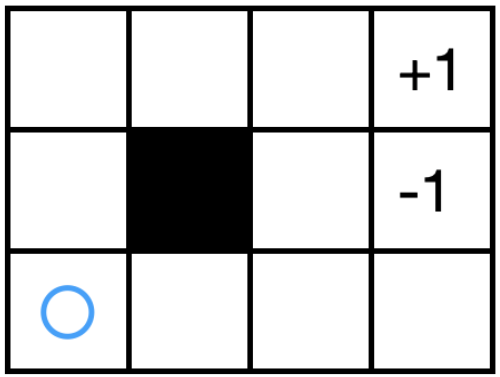
Remark

The algorithm is guaranteed to terminate in a finite number of steps and give the optimal policy.
The computational time is at most $O(|X|^2|A|) + O(|X|^3)$ per iteration.

Markov Decision Process: examples

Example (GRIDWORLD)

Consider a robot in a 3×4 grid with one inaccessible cell, two terminal cells with reward ± 1, and reflecting walls.
Available actions are $a_t \in \{T, B, L, R\}$, non-terminal cell give a reward $r \leq 0$. Expected utility function is discounted by factor $\delta < 1$.
Robot's actions are not deterministic, with probability $p = 0.8$ the action is executed, otherwise the robot moves uniformy in the orthogonal directions. Moving towards a wall results in resting in place.



Markov Decision Process: examples

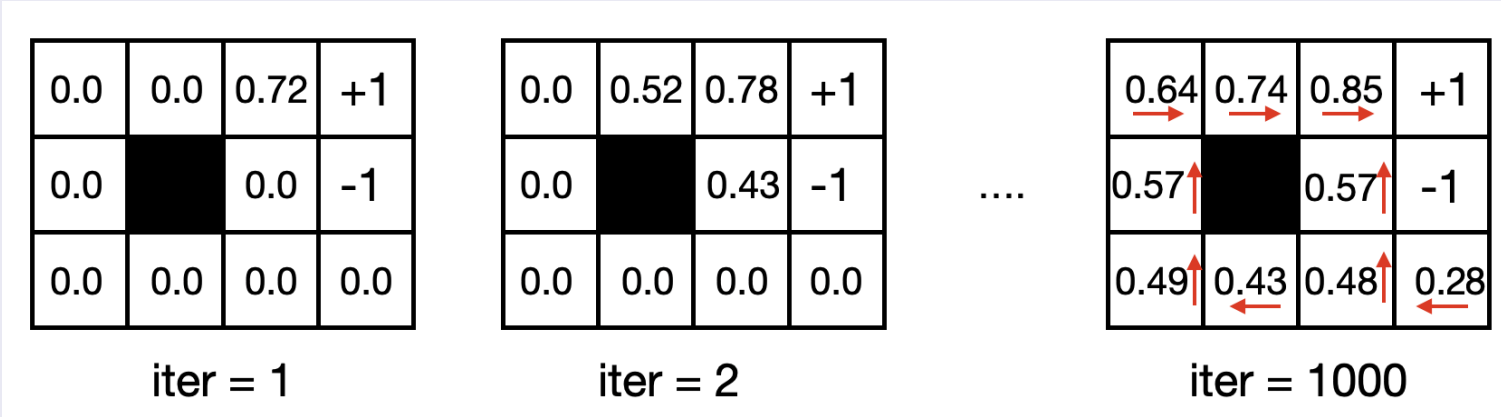
Example (cont'd)

Using value iteration we initialize $V_{(0)}(x) = 0$ for all non-terminal states and compute $\forall i \geq 0$

$$V_{(i+1)}(x) = \max_{a \in \{T,B,L,R\}} \left[r + \delta \sum_{x' \in X} p_{xx'}(a) V_{(i)}(x') \right].$$

Then find convergence for $i \to \infty$.

For instance, setting $r = 0$ and $\delta = 0.9$, we obtain



Markov Decision Process: examples

Example (Cleaning Robot)

A recycling robot can

1. actively search for empty cans over a certain period ("search"),
2. wait statically for someone to bring a can ("wait")
3. head back to home base to recharge ("recharge").

Actions are $a \in \{S, W, R\}$.

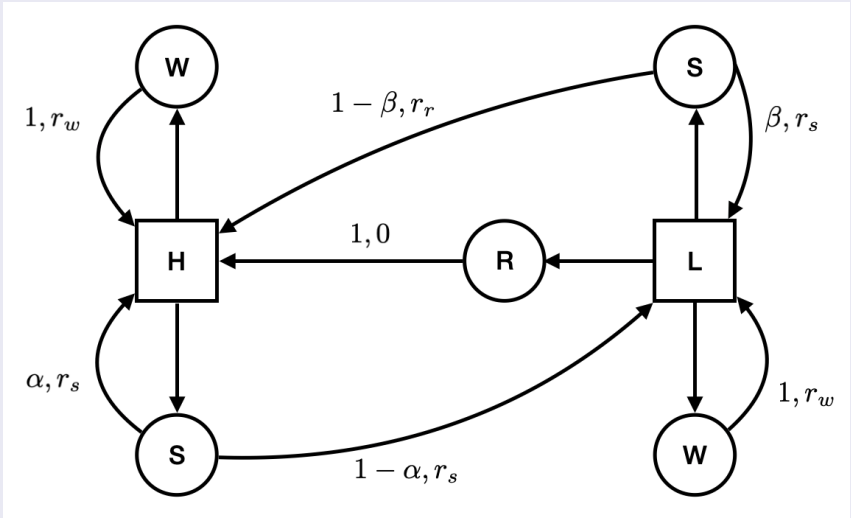
States of the system: robot's battery level "high" or "low", i.e. $x \in \{H, L\}$. Transitions from states occur with known probabilities $0 \leq \alpha, \beta \leq 1$ (see diagram).

Rewards: $r_s > r_w > 0$ and $r_r \ll 0$ (running out of battery while searching).

Find the optimal strategy assuming discount $\delta < 1$.

Markov Decision Process: examples

Example (cont'd)



The optimal value is given by the stationary Bellman equation

$$V(H) = \max \begin{cases} r_w + \delta V(H) & a = W \\ r_s + \delta \left[\alpha V(H) + (1-\alpha) V(L)\right] & a = S \end{cases}$$

$$V(L) = \max \begin{cases} \delta V(H) & a = R \\ r_w + \delta V(L) & a = W \\ \beta \left[r_s + \delta V(L)\right] + (1-\beta)\left[r_r + \delta V(H)\right] & a = S \end{cases}$$

Markov Decision Process: reinforcement learning

Reinforcement Learning is useful when we are able to simulate the Markov Chain and measure rewards but we don't know the underlying transition probabilities (**model free**).

Episode: a sample path of a MDP generated from an initial condition x_0 under a policy μ.

Offline learning: value function $V_{(i)}(x)$ is updated for each state x at the end of each episode i simultaneously.

Online learning: value function $V_{(i)}(x)$ is updated for each visited state x in the order resulting from the i-th episode (asynchronous).

Two main ingredients:

- Policy Improvement, $\mu_{(i+1)}(x) \in \arg \mathcal{B}[V_{(i)}](x), \qquad \forall x$
- Policy Evaluation, $V_{(i+1)} = r^{\mu} + \delta P^{\mu} V_{(i)}$

to be implemented model-free.

Markov Decision Process: reinforcement learning

Monte Carlo sampling policy evaluation: at the end of each episode i, if state x is visited, the number $n_{(i)}(x)$ of visits and the value function $V_{(i)}(x)$ are updated as follows

$$\begin{aligned} n_{(i)}(x) &= n_{(i-1)}(x) + 1 \\ V_{(i)}(x) &= V_{(i-1)}(x) + \frac{1}{n_{(i)}(x)} \left[\tilde{R}_{(i)}(x) - V_{(i-1)}(x) \right] \end{aligned}$$

where

$$\tilde{R}_{(i)}(x) = r(x, x_1 = \mu(x)) + \delta r(x_1, x_2 = \mu(x_1)) + \cdots + \delta^{T_i(x)} r\left(x_{T_i(x)}\right)$$

is the observed cumulative reward after visiting state x in episode i.

The value $V_{(i)}(x)$ tends to $V^{\mu}(x)$ in the limit $n_{(i)}(x) \to \infty$.

Remark

Simple but offline learning. Estimates are unbiased but have big variance.

Markov Decision Process: reinforcement learning

Time Difference policy evaluation: at every visit of state x, a value function $V(x)$ is updated as follows

$$\tilde{V}^{\mu}(x) = \tilde{V}^{\mu}(x) + \alpha_t(x)\left[r(x,\mu(x)) + \delta\tilde{V}^{\mu}(x') - \tilde{V}^{\mu}(x)\right]$$

where $x' = \mu(x)$.

Under the (Robbins-Munro) conditions:

- state x is visited infinitely often
- the learning rate $\alpha_t(x)$ satisfies $\forall x$

$$\sum_{t=0}^{\infty}\alpha_t(x) = +\infty, \qquad \sum_{t=0}^{\infty}\alpha_t(x)^2 < +\infty$$

the algorithm converges to the correct expected value $V^{\mu}(x)$.

Remark

The learning is online. Estimates have lower variance but possibly biased.

Markov Decision Process: reinforcement learning

How can we implement policy improvement in a model free way?

Define the ***action-value function*** $Q^{\mu}(x, a)$ such that

$$Q^{\mu}(x, a) = r(x, a) + \delta \sum_{x' \in X} p_{xx'}(a) V^{\mu}(x'), \qquad \forall x, \forall a \in A(x)$$

Therefore $V^{\mu}(x) = Q^{\mu}(x, \mu(x))$ and, for an optimal policy,

$$V(x) = \max_{\mu} Q^{\mu}(x, \mu(x))$$

The Bellman equation for V implies a recursive equation for the optimal action-value function

$$Q(x, a) = r(x, a) + \delta \sum_{x' \in X} p_{xx'}(a) \max_{a'} Q(x', a')$$

Markov Decision Process: reinforcement learning

Q-learning (Watkins, 1989) is a learning algorithm based on the update:

- *Input*: initial function $Q_{(0)}(x, a)$, $\forall x \in X$, $\forall a \in A$; initial state x.
- *Repeat:* based on some convergence criterion,
  - choose a from $A(x)$ using a policy derived from $Q_{(i)}(x, a)$ (e.g. softmax $\Pr[a|x] \propto e^{\beta Q_{(i)}(x,a)}$);
  - draw (observe) new state x'
  - update action-value function

$$Q_{(i+1)}(x, a) = Q_{(i)}(x, a) + \alpha \left\{ r(x, a) + \delta \max_{a'} Q_{(i)}(x', a') - Q_{(i)}(x, a) \right\}$$

  - set $i \leftarrow i + 1$;
- *Output*: (optimal) action-value function $Q_{(i)}(x, a)$.

Remark

The algorithm asymptotically converges to the optimal action-value function. It is model-free, can be used on a training set of data.

Markov Decision Process: reinforcement learning

Sarsa (Sutton, 1996) eliminates the max operation in favor of next action a' chosen according to some rule derived from Q (e.g. same as for a):

- *Input*: initial function $Q_{(0)}(x, a)$, $\forall x \in X$, $\forall a \in A$; initial state x.
- *Repeat:* based on some convergence criterion,
  - choose a from $A(x)$ using a policy derived from $Q_{(i)}(x, a)$ (e.g. softmax $\Pr[a|x] \propto e^{\beta Q_{(i)}(x,a)}$);
  - draw (observe) new state x'
  - choose a' from $A(x')$ using a policy derived from $Q_{(i)}(x', a')$
  - update action-value function

$$Q_{(i+1)}(x, a) = Q_{(i)}(x, a) + \alpha \left\{ r(x, a) + \delta Q_{(i)}(x', a') - Q_{(i)}(x, a) \right\}$$

  - set $i \leftarrow i + 1$;
- *Output*: (optimal) action-value function $Q_{(i)}(x, a)$.

Remark

The algorithm asymptotically converges to the optimal action-value function if the policy is well-behaved (e.g. softmax).